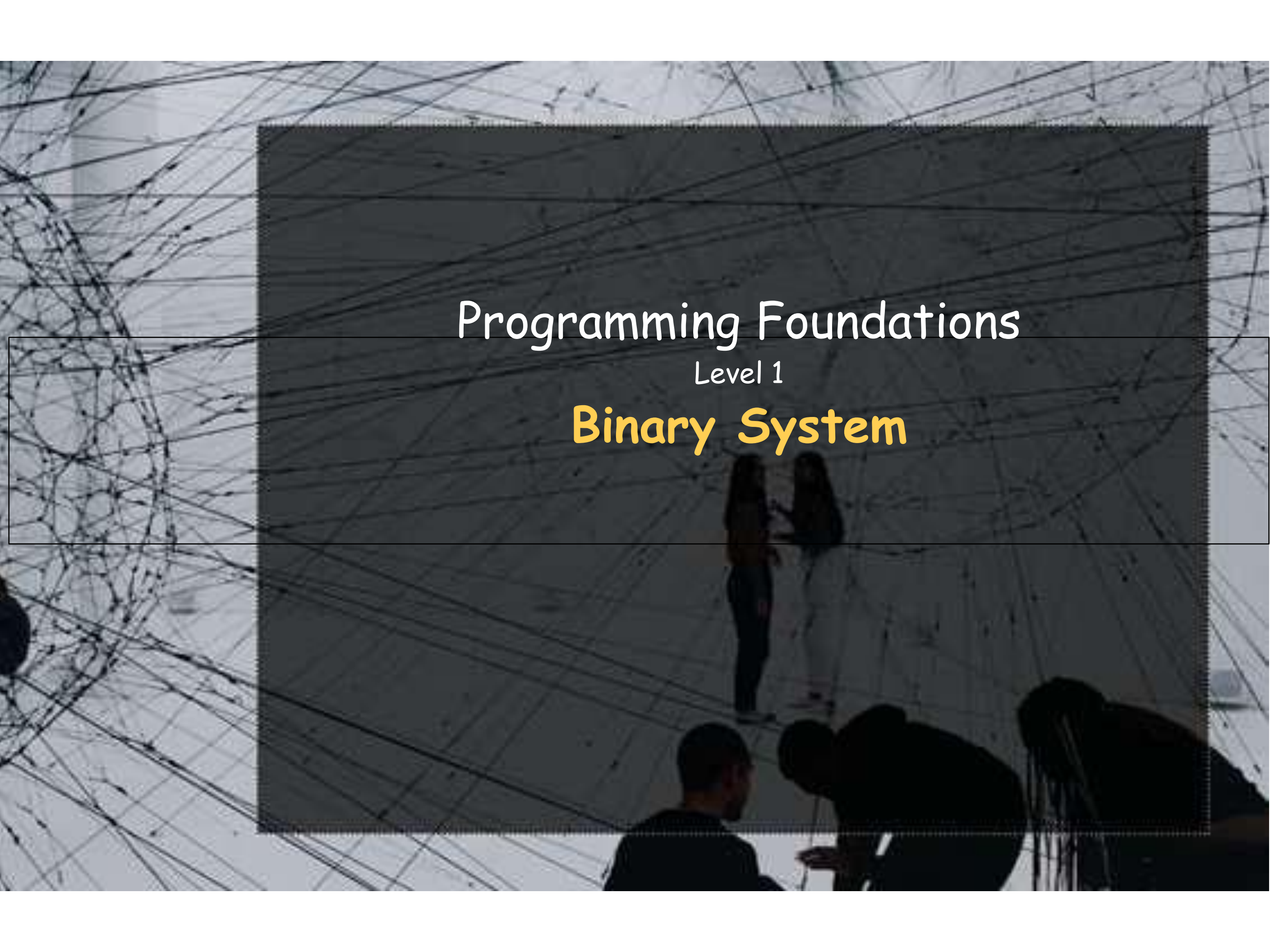




Programming Foundations Level 1

The background of the slide features a complex, abstract pattern of thin, dark lines crisscrossing over a lighter, textured surface. In the lower portion of the image, the silhouettes of several people are visible, appearing to be in a collaborative setting, possibly a meeting or a workshop, with some individuals leaning over a table or screen.

Programming Foundations

Level 1

Binary System

Understanding the Binary System in Computers

Computers are electronic devices that work entirely using electricity — signals that are either **ON** or **OFF**.

Because of this, they can only understand two states: **1 (ON)** and **0 (OFF)**.

This fundamental concept is the basis of everything a computer does — numbers, letters, images, and sounds are all represented in binary form.

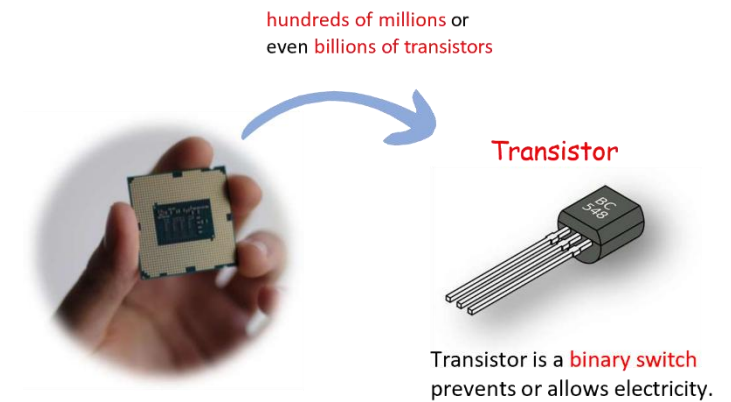
Understanding the binary system is the first step toward mastering computer logic and programming.

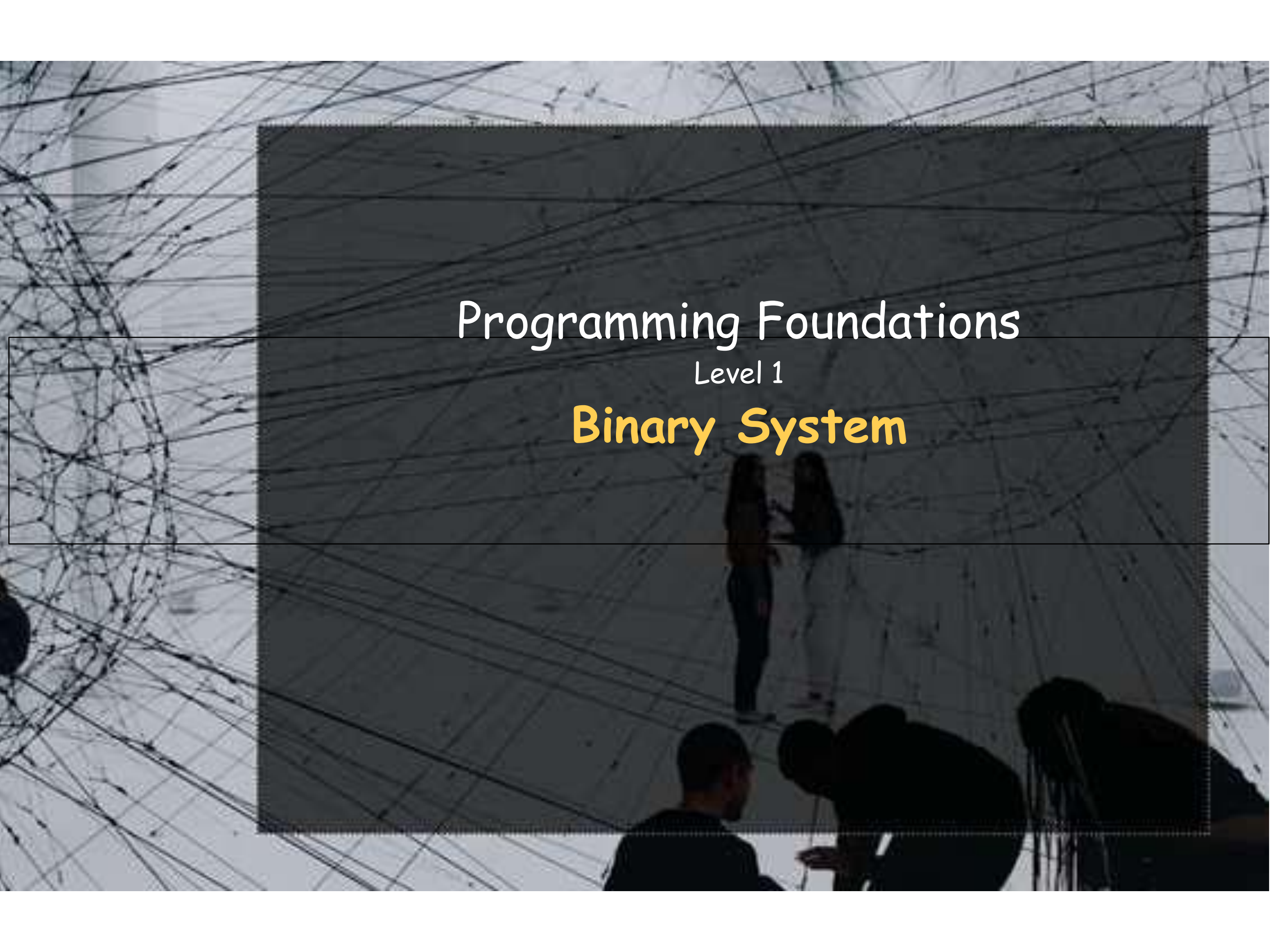
How Computer Works?

- A **computer** processes information using **electrical signals**.
- It understands only **0s and 1s** (binary digits).
- These digits represent **electrical states** — **Off (0)** or **On (1)**.
- Billions of these tiny electrical switches called **transistors** form the **CPU** and handle all computations.
- Inside the **CPU (Central Processing Unit)**, there are **hundreds of millions — or even billions — of transistors**.
- A **transistor** is a microscopic **binary switch** that either **allows** or **blocks** the flow of electricity.
- Every calculation, text, sound, or image you see on your screen is made possible by these transistors **switching ON and OFF billions of times per second**.
- 💡 **Example:**
When you press a key on your keyboard, your **CPU** processes the electrical signal (a pattern of 0s and 1s) to decide what to display.

How Computer Works?

- Computer is an electronic device.
- It deals with electricity On/Off
- It only understands **0's** (Off) and **1's** (On).
- Everything you see, hear and read on the screen reaches you as the result of "on" or "off" billions of times.
- Computer uses Binary System to represent numbers, letters, commands, images and sounds.



The background of the slide is a dark, abstract image. It features a dense, intricate network of thin, light-colored lines that resemble a complex web or a neural network. In the lower portion of the image, there are dark silhouettes of several people. Two people are standing in the center, facing each other, while others are seated or crouching in the foreground, looking towards the center. The overall tone is technical and futuristic.

Programming Foundations

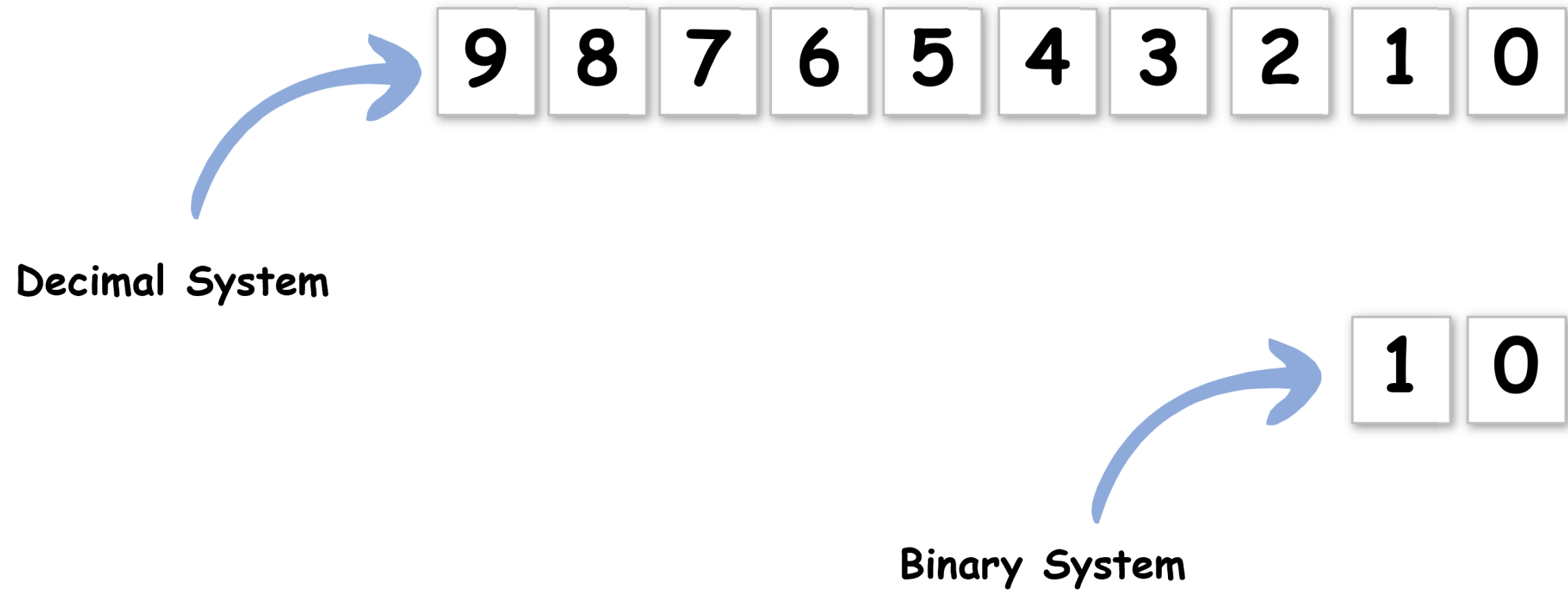
Level 1

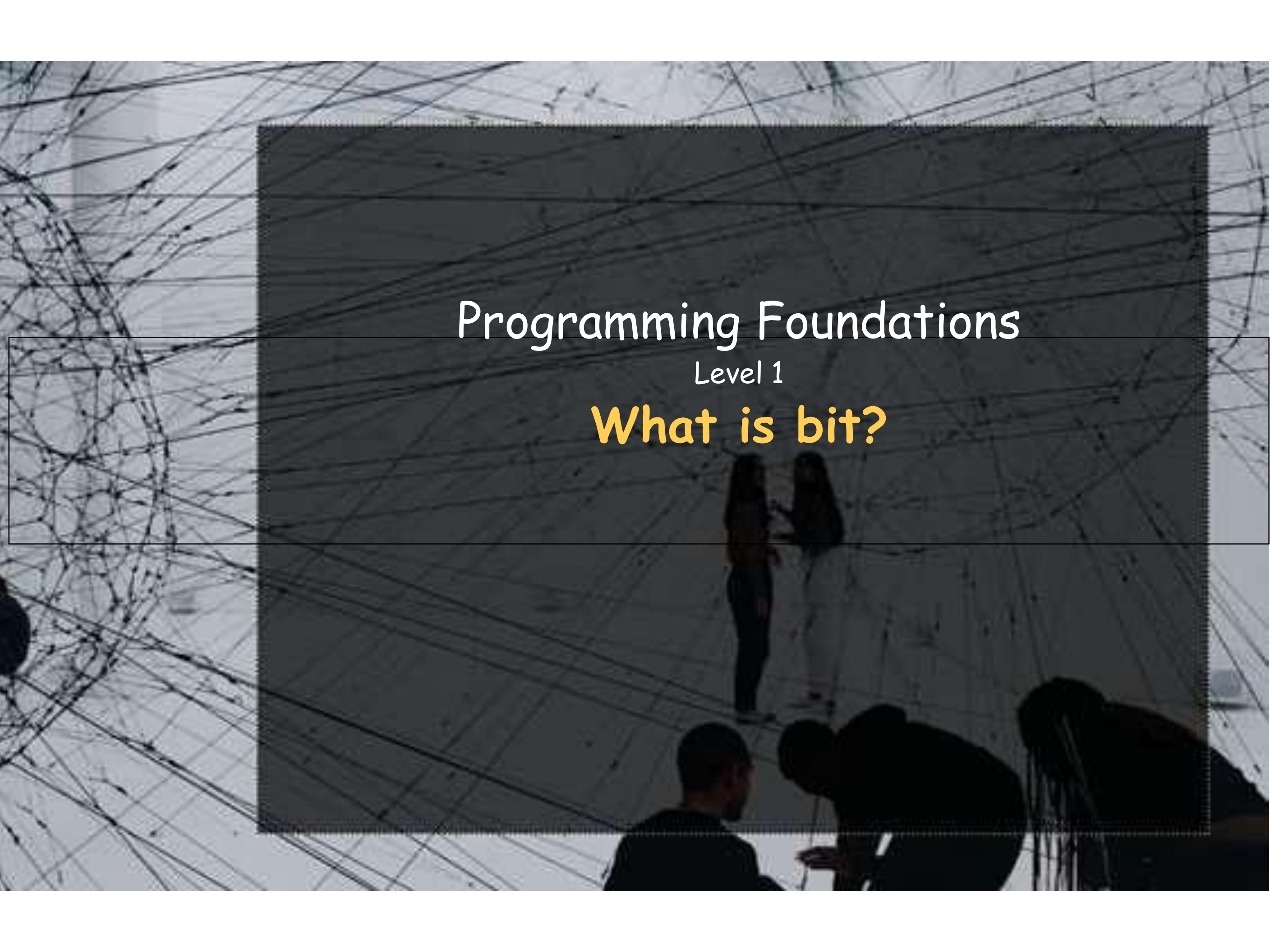
Binary System

The Binary System

- The **binary system** is a **base-2** numbering system.
- It uses only **two digits**: **0** and **1**.
- Every binary position represents a **power of 2** (1, 2, 4, 8, 16, 32, 64, 128 ...).

Decimal vs Binary?



The background of the slide features a complex, abstract pattern of thin, dark lines crisscrossing over a lighter, textured surface. In the lower portion of the image, there are dark silhouettes of several people, some standing and some leaning over, suggesting a collaborative work environment or a group discussion. The overall aesthetic is technical and modern.

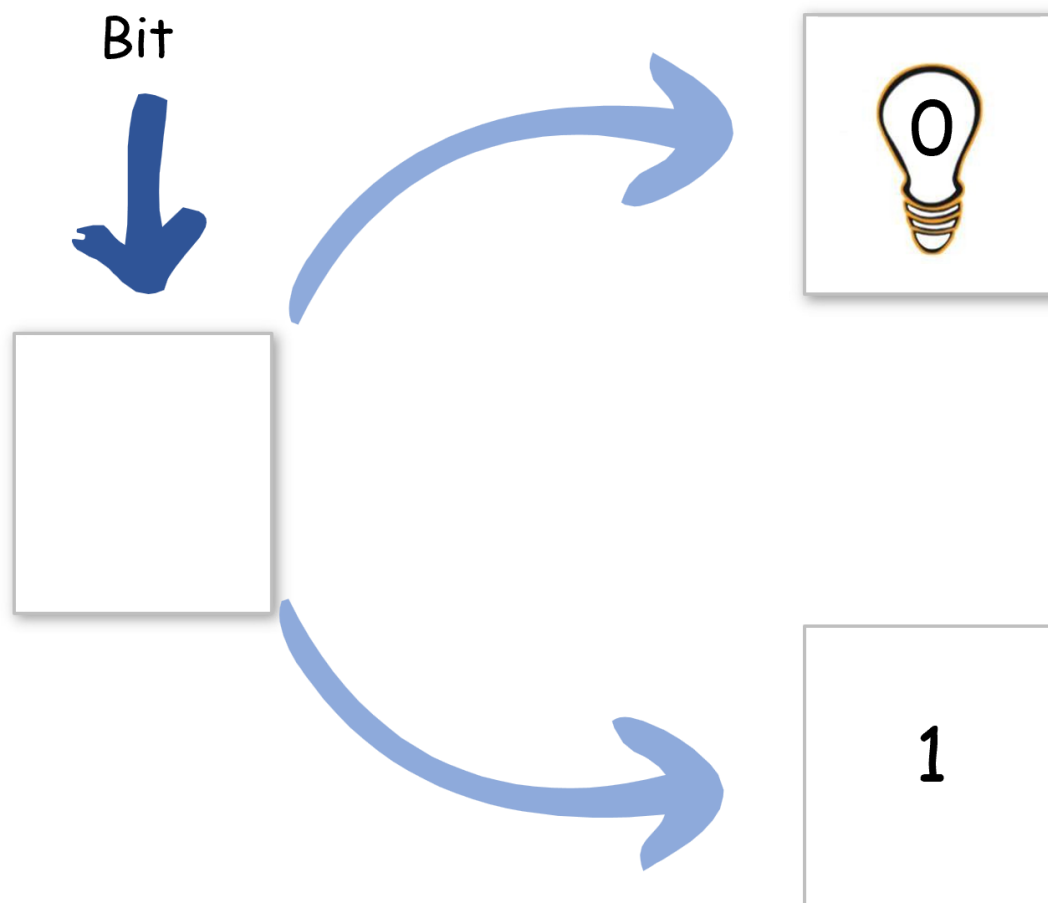
Programming Foundations

Level 1

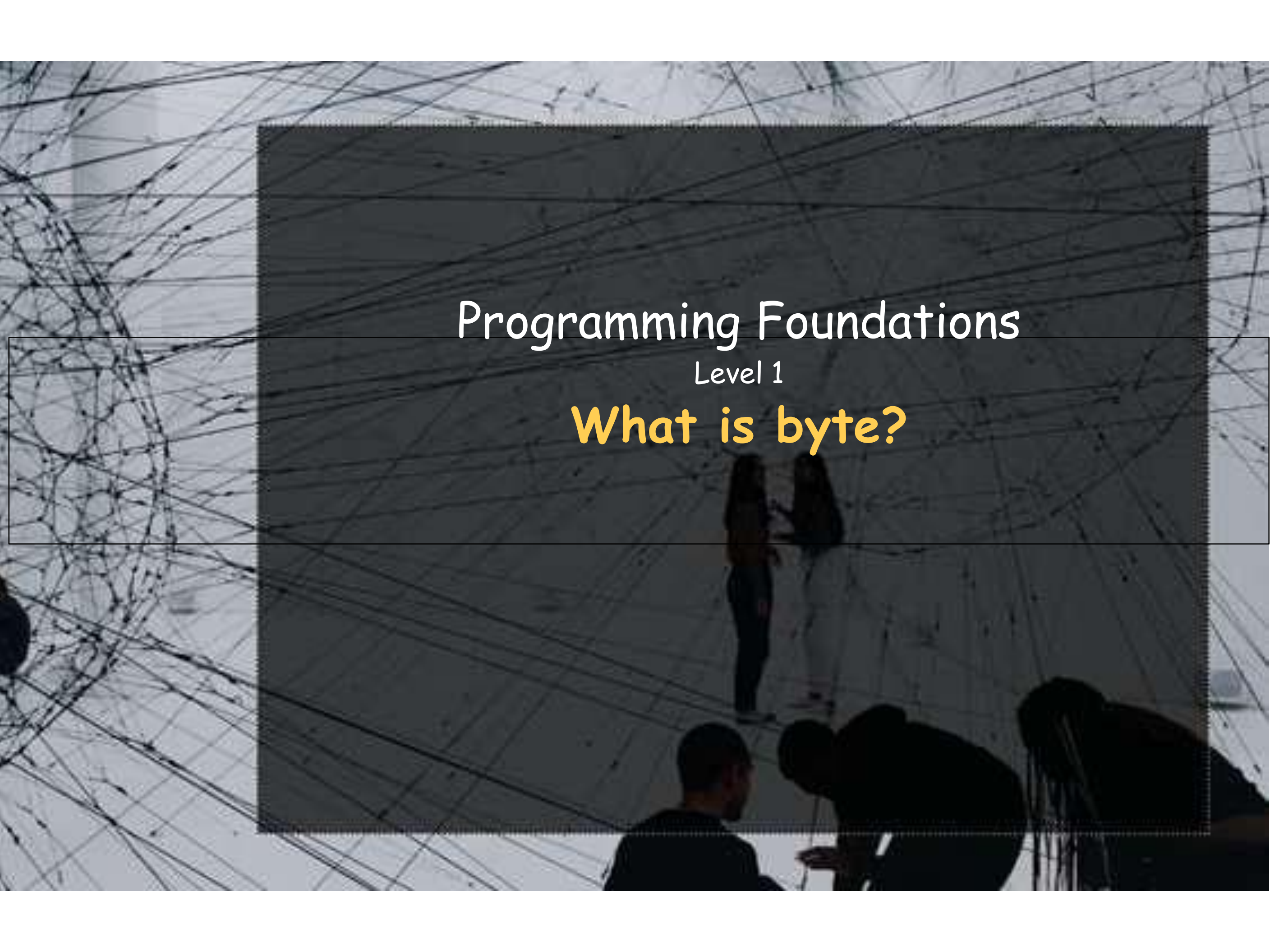
What is bit?

What is Bit?

Remember! .. Computer is an electronic device...it deals with electricity only.
Bit = "Binary Digit" → the smallest piece of information (either 0 or 1).



A bit is a **binary digit**, the smallest increment of data on a computer



Programming Foundations

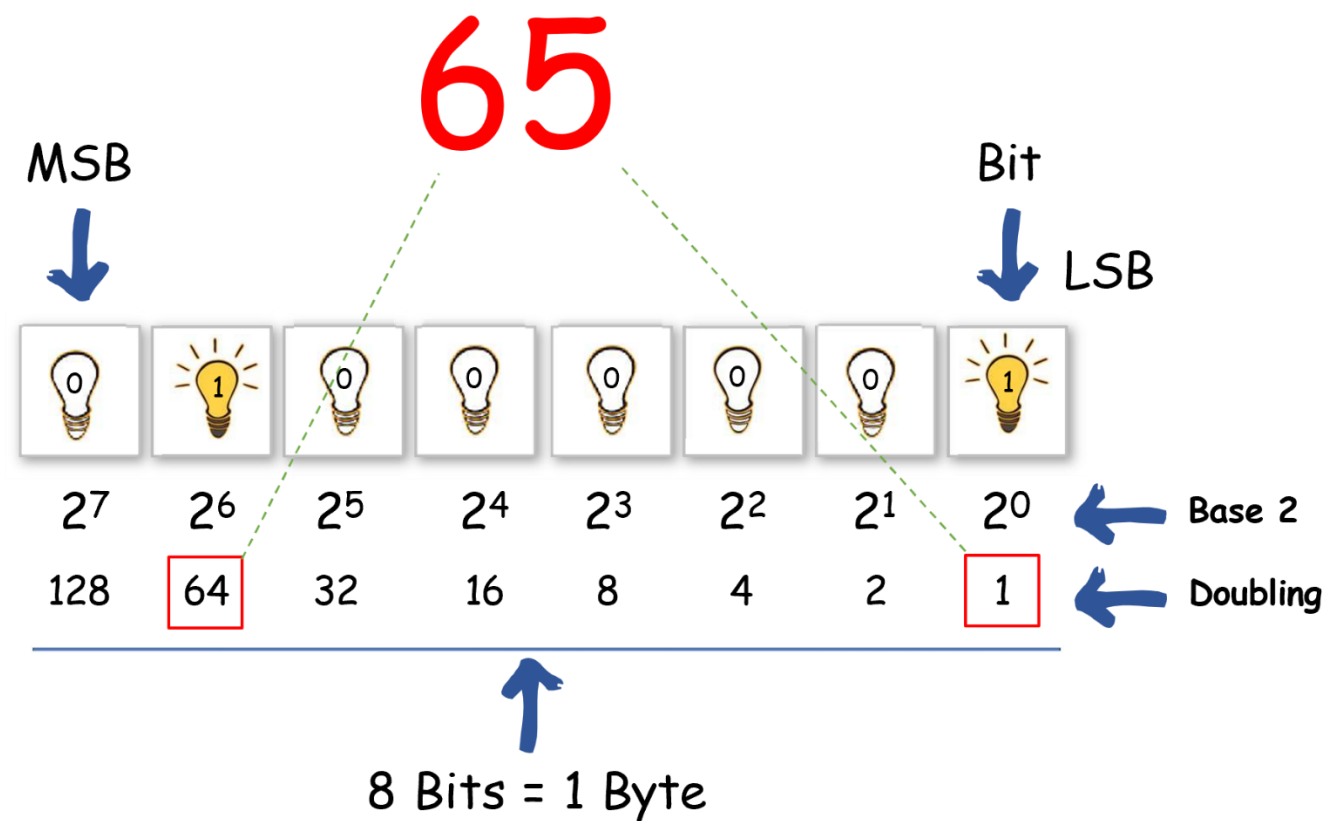
Level 1

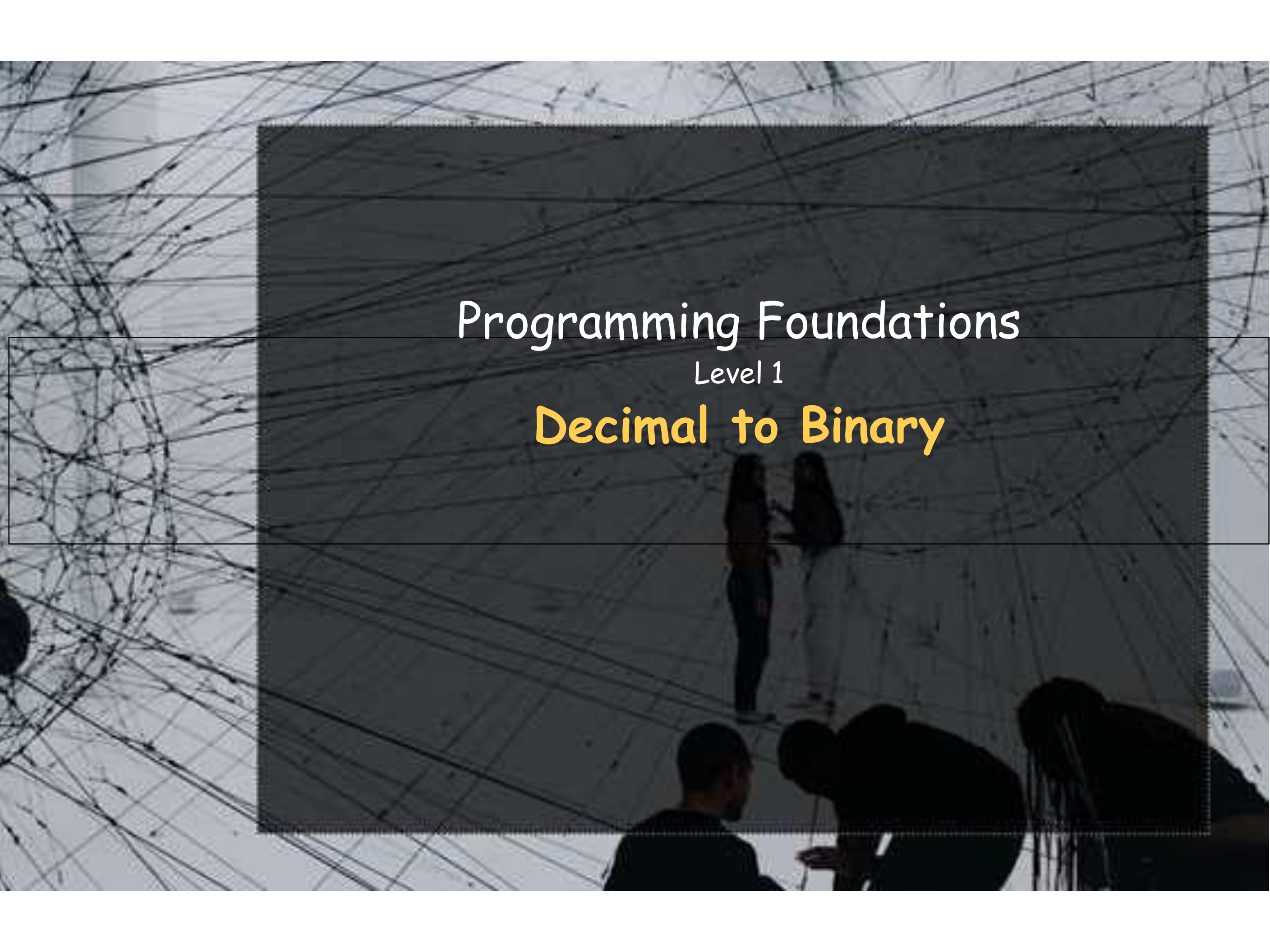
What is byte?

What is Byte?

Byte = group of 8 bits.

Each bit in a byte has a positional value:



The background of the slide is a dark, abstract image. It features a dense, intricate network of thin, light-colored lines that resemble a complex web or a neural network. In the lower portion of the image, there are dark silhouettes of several people. Two people are standing in the center, facing each other, while others are visible in the foreground, some appearing to be looking at a screen or device. The overall aesthetic is technical and modern.

Programming Foundations

Level 1

Decimal to Binary

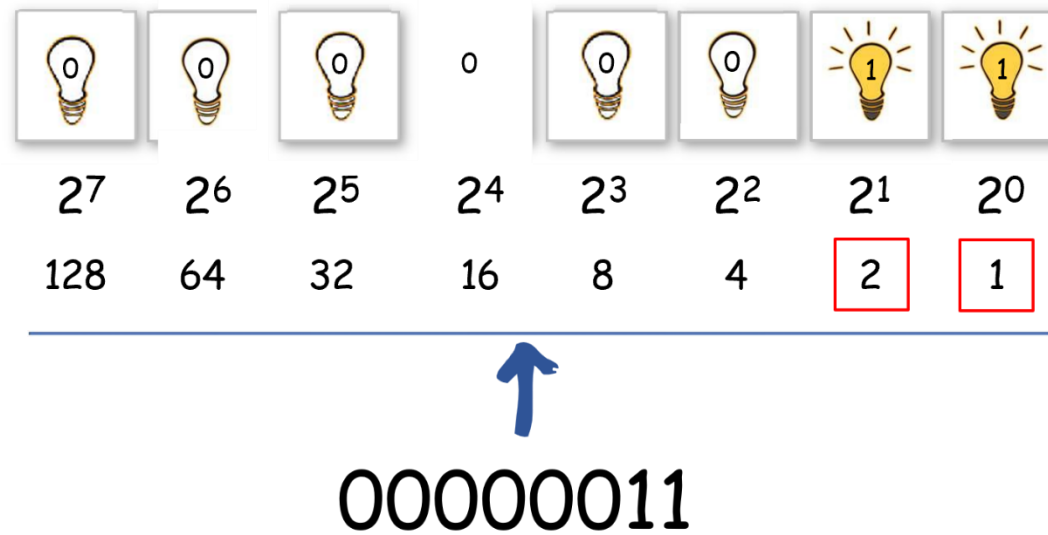
Converting Between Decimal and Binary

Decimal → Binary:

Divide the decimal number by 2 repeatedly and note the remainders (bottom to top).

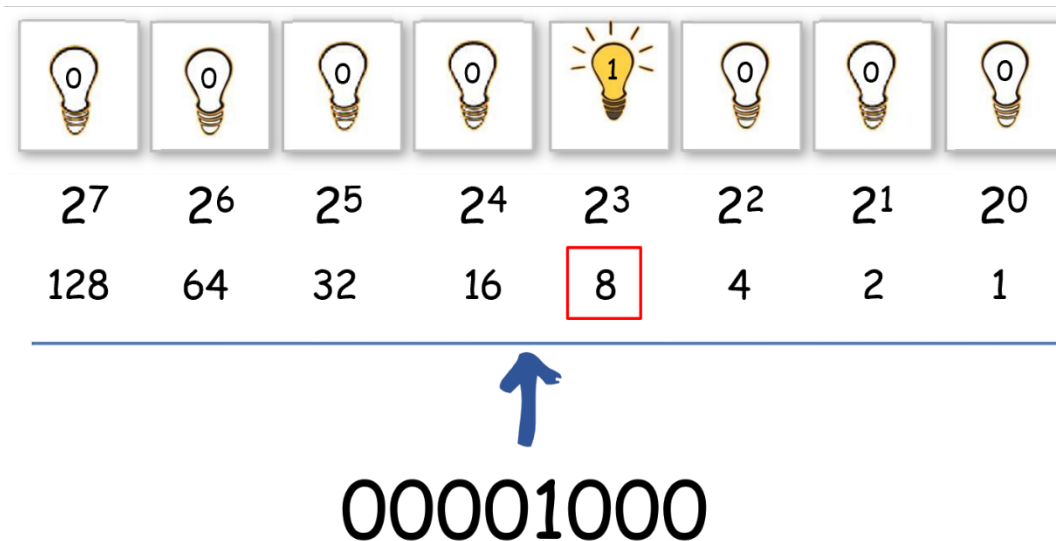
Represent this in binary...

3



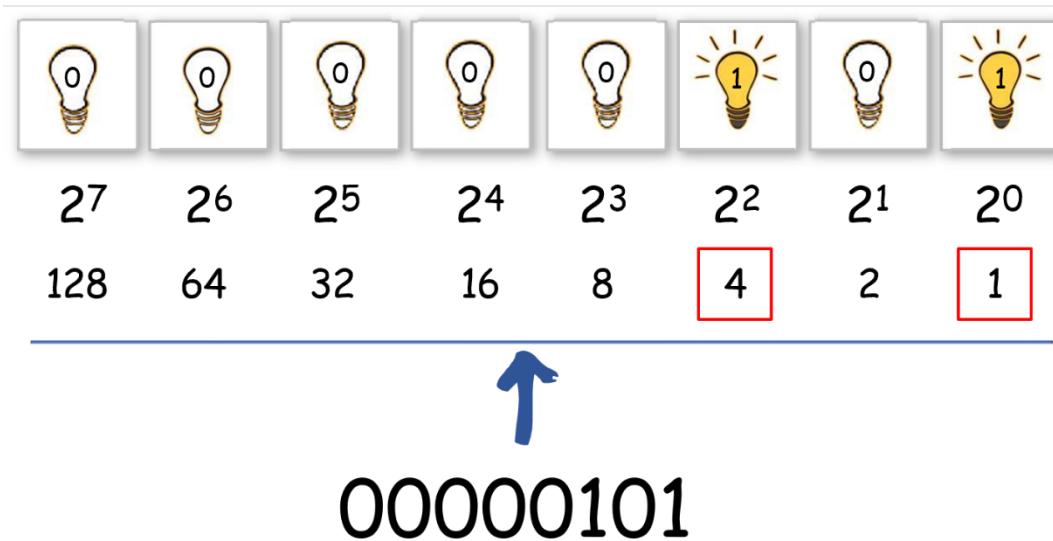
Represent this in binary...

8



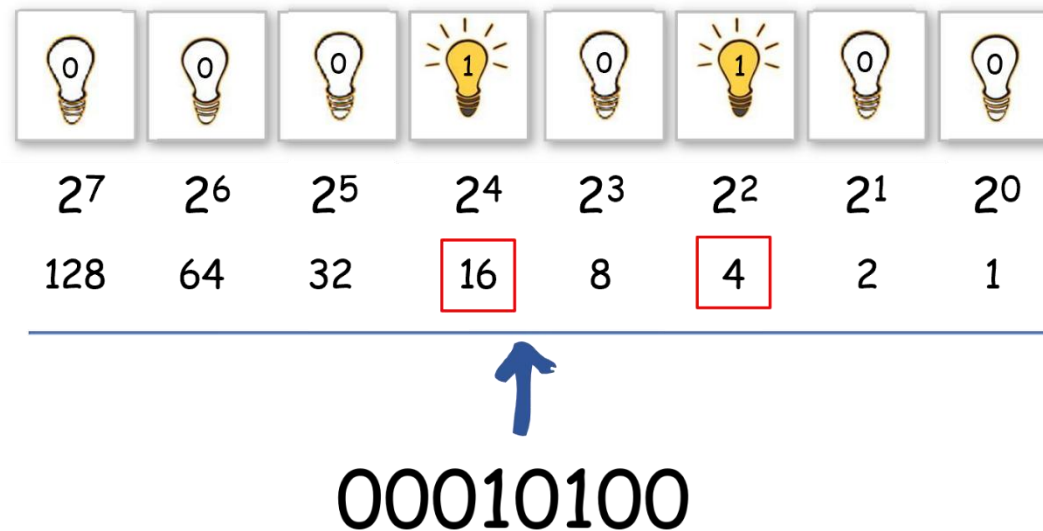
Represent this in binary...

5



Represent this in binary...

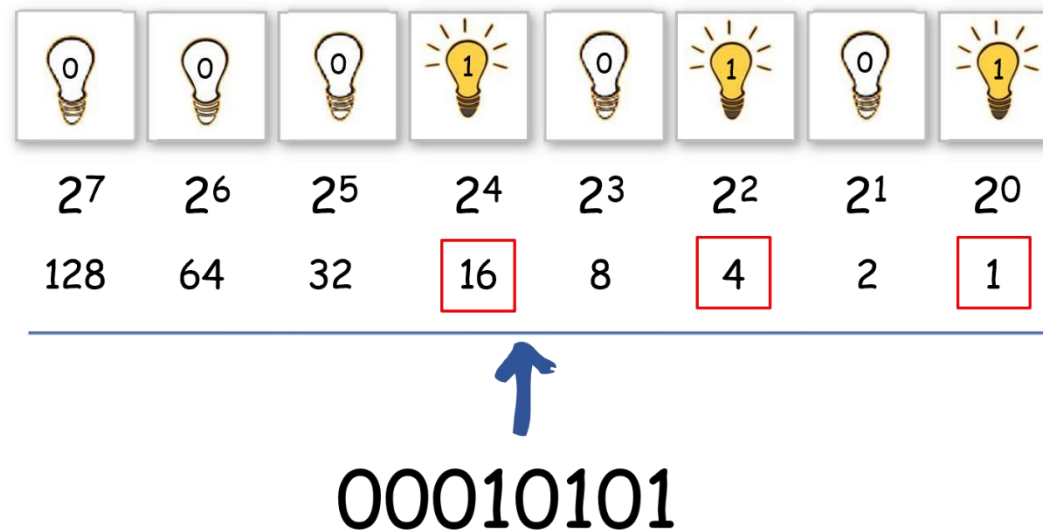
20

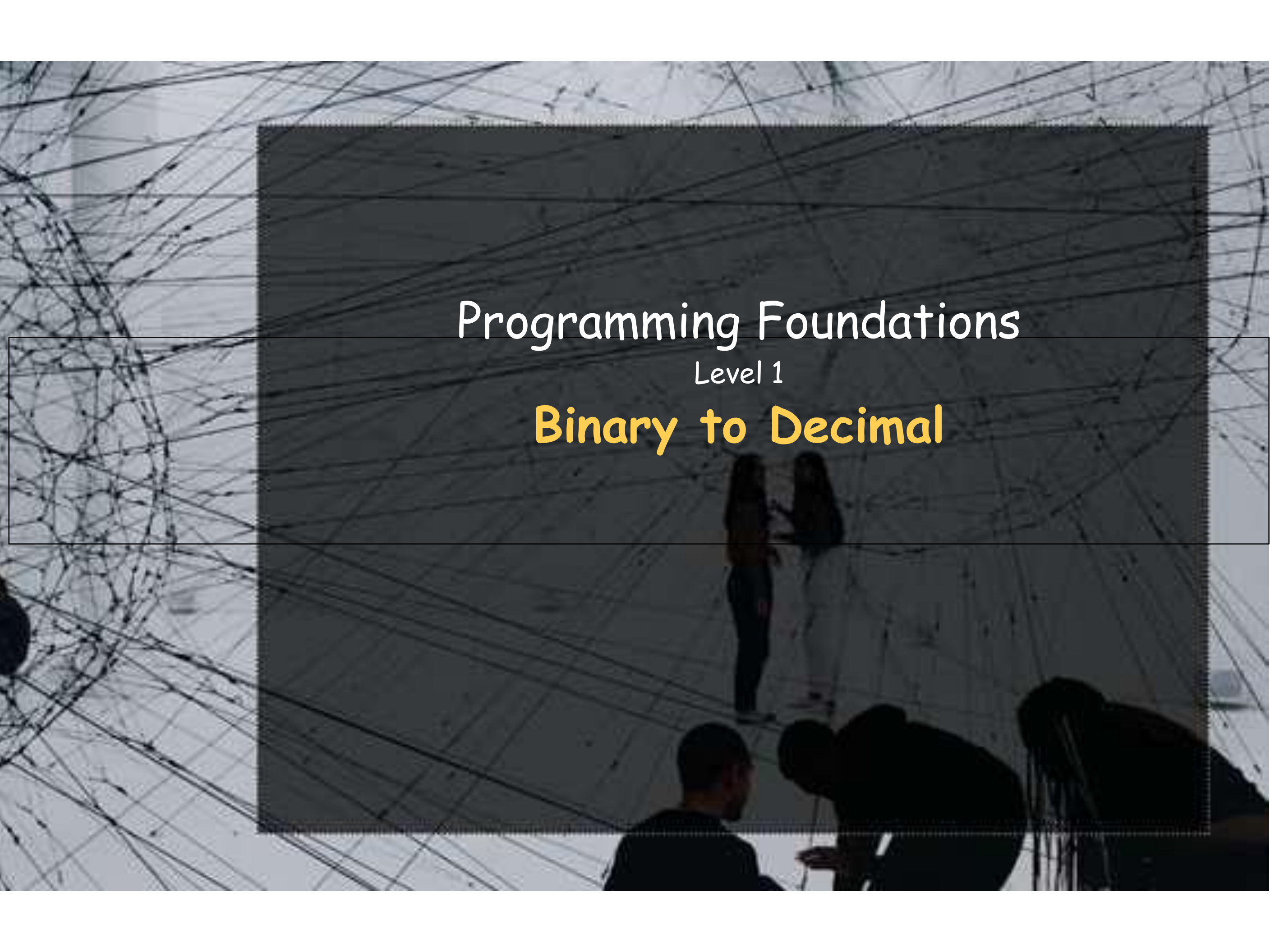


Represent this in binary...

21

Only one way
to represent Data





Programming Foundations

Level 1

Binary to Decimal

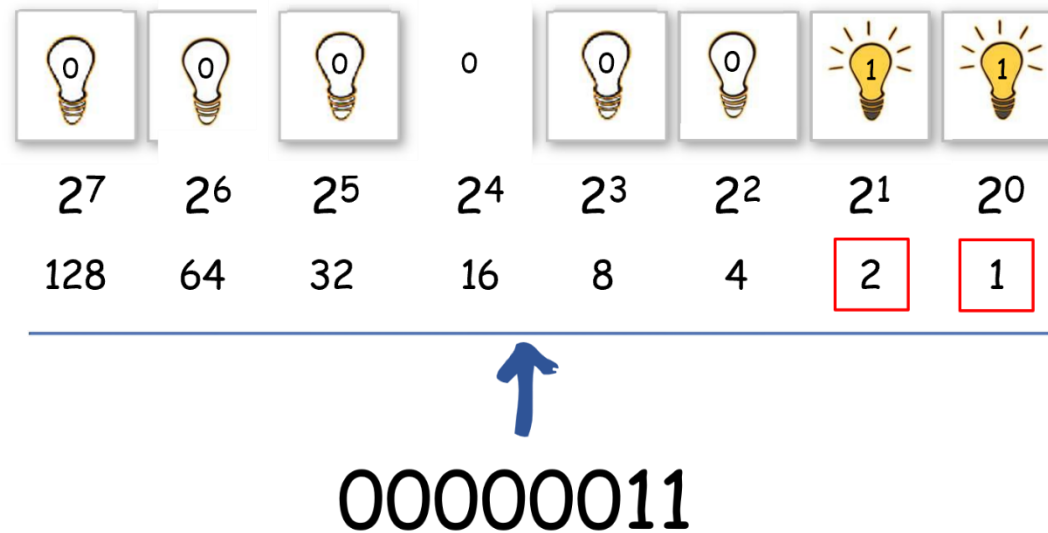
Converting Between Decimal and Binary

Binary → Decimal:

Multiply each bit by its positional value and add them.

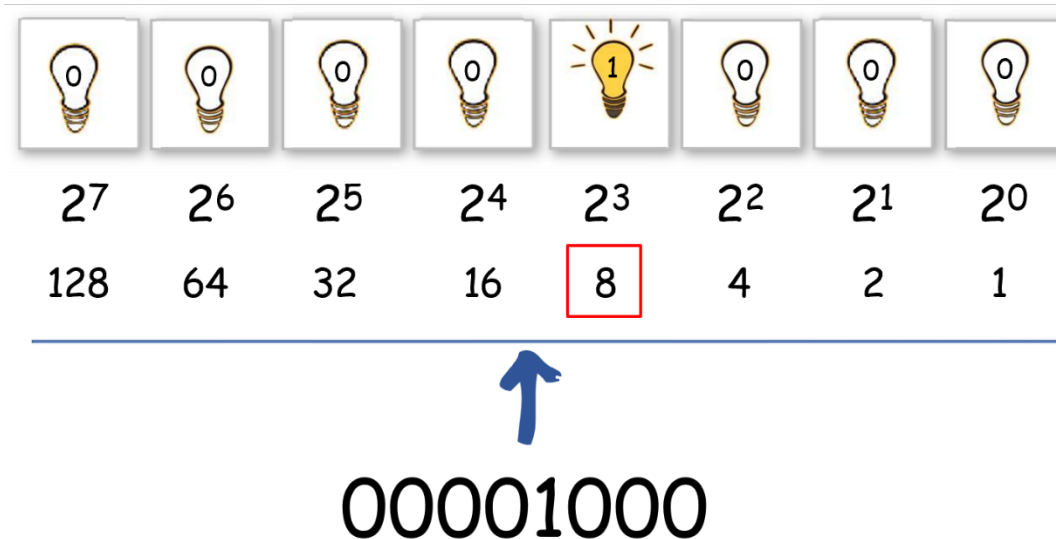
Represent this in decimal...

3



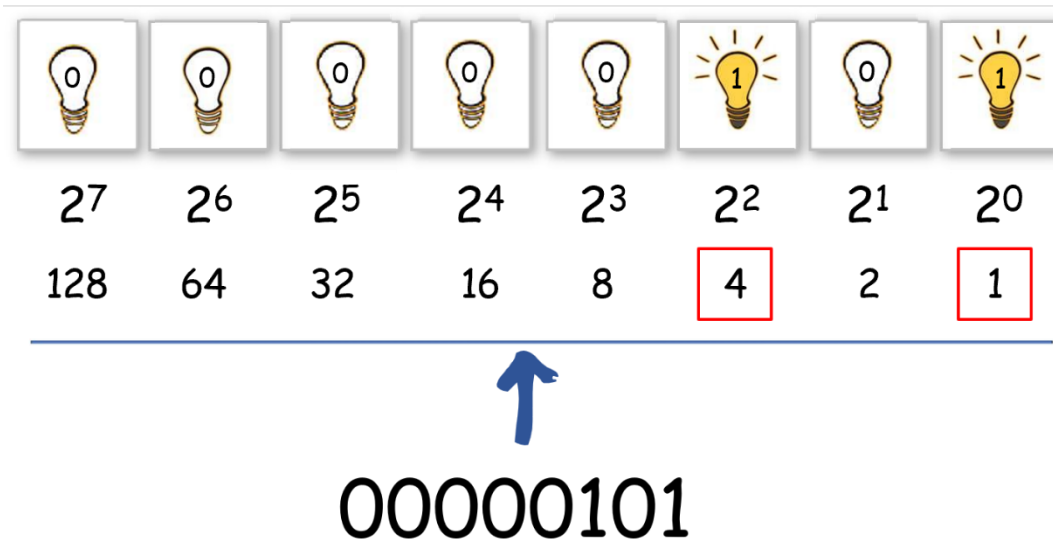
Represent this in decimal...

8



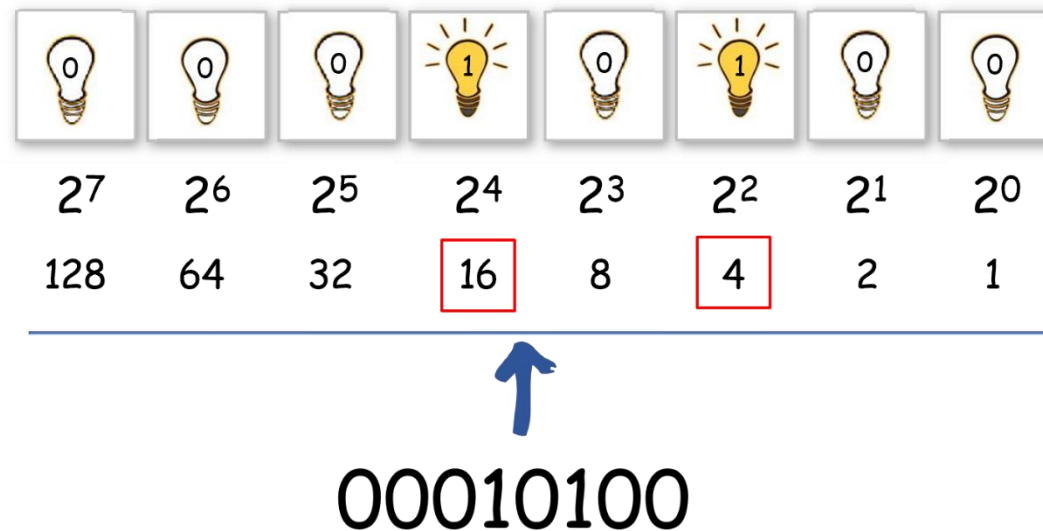
Represent this in decimal...

5



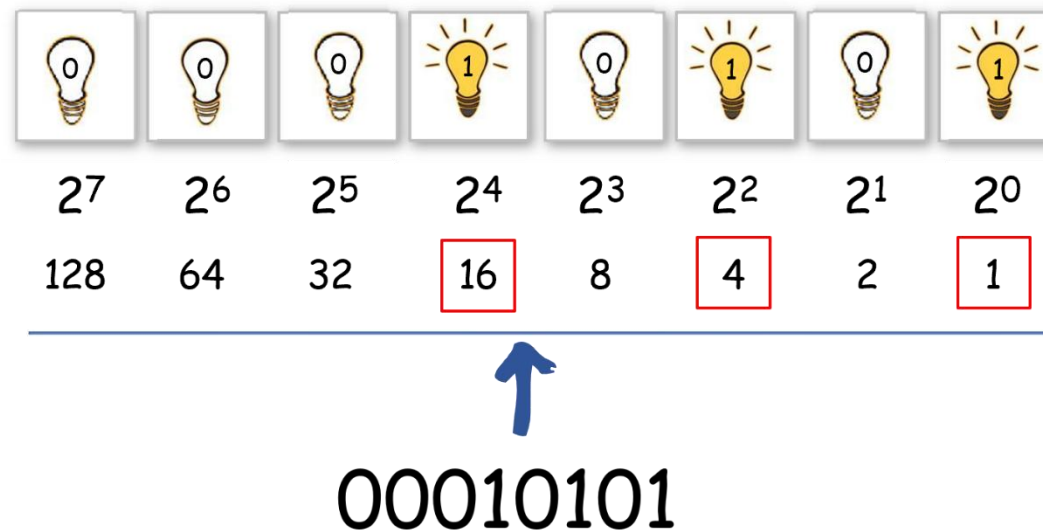
Represent this in decimal...

20



Represent this in decimal...

21



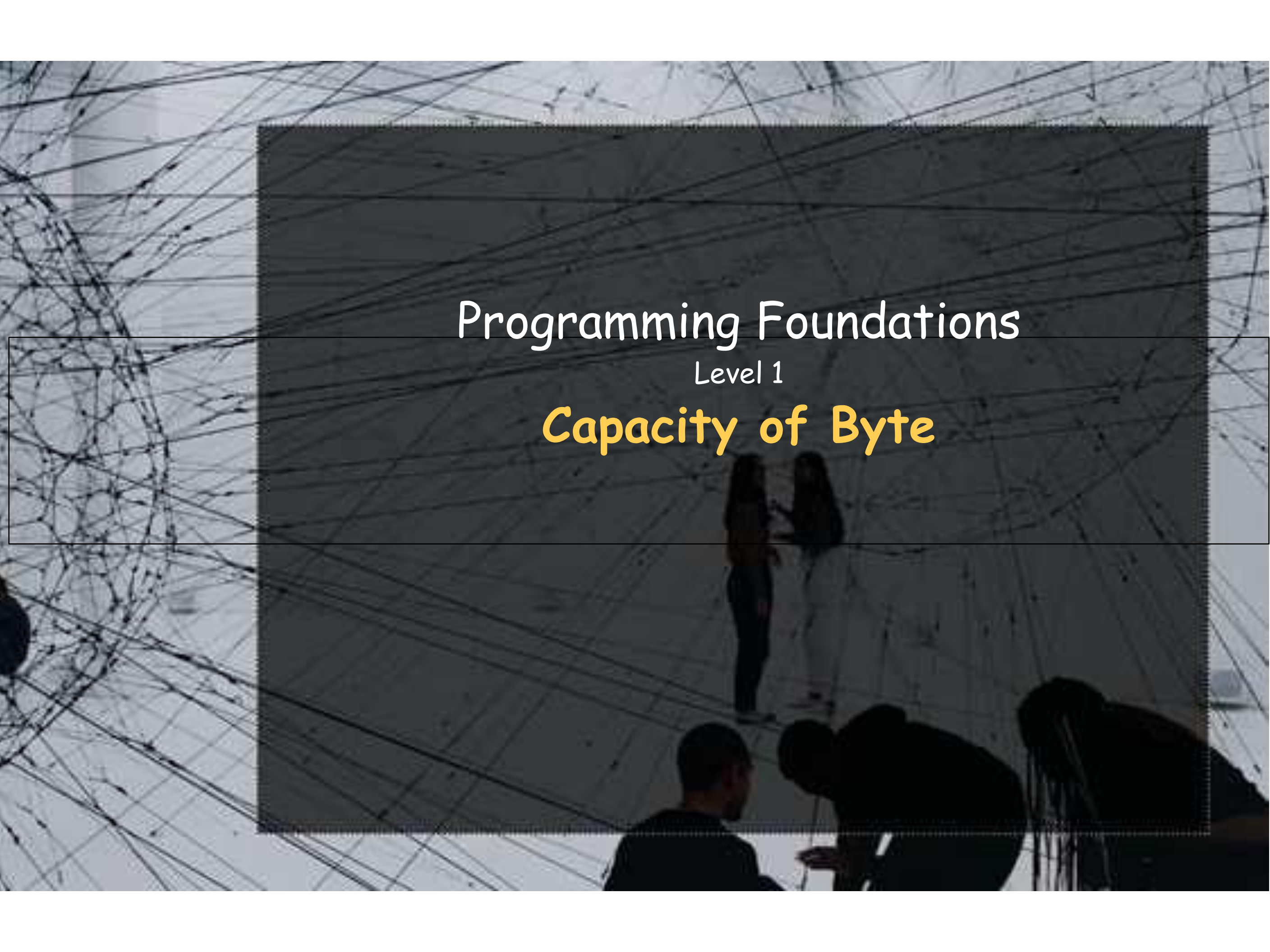
"Only One Way" to Represent a Number in Binary

In **binary**, every non-negative integer has **exactly one** representation (if you ignore **leading zeros**).

This means there is **one unique set of bits** (MSB...LSB) whose weighted sum equals the number.

You **cannot** choose a different combination of bit positions to get the **same** number — each power of two (1, 2, 4, 8, 16, ...) is either **used once** or **not used**.

So in the above example the number 20 only comes from 4 + 16 bits, while in decimal system 20 can come from 10 + 10 , or 18 + 2 , or 15 + 5 ...etc.

The background of the slide features a complex, abstract pattern of thin, dark lines crisscrossing over a lighter, textured surface. In the lower portion of the image, there are dark silhouettes of several people, some standing and some leaning over, suggesting a collaborative work environment or a classroom setting. A large, dark, semi-transparent rectangular box is centered on the slide, serving as a backdrop for the text.

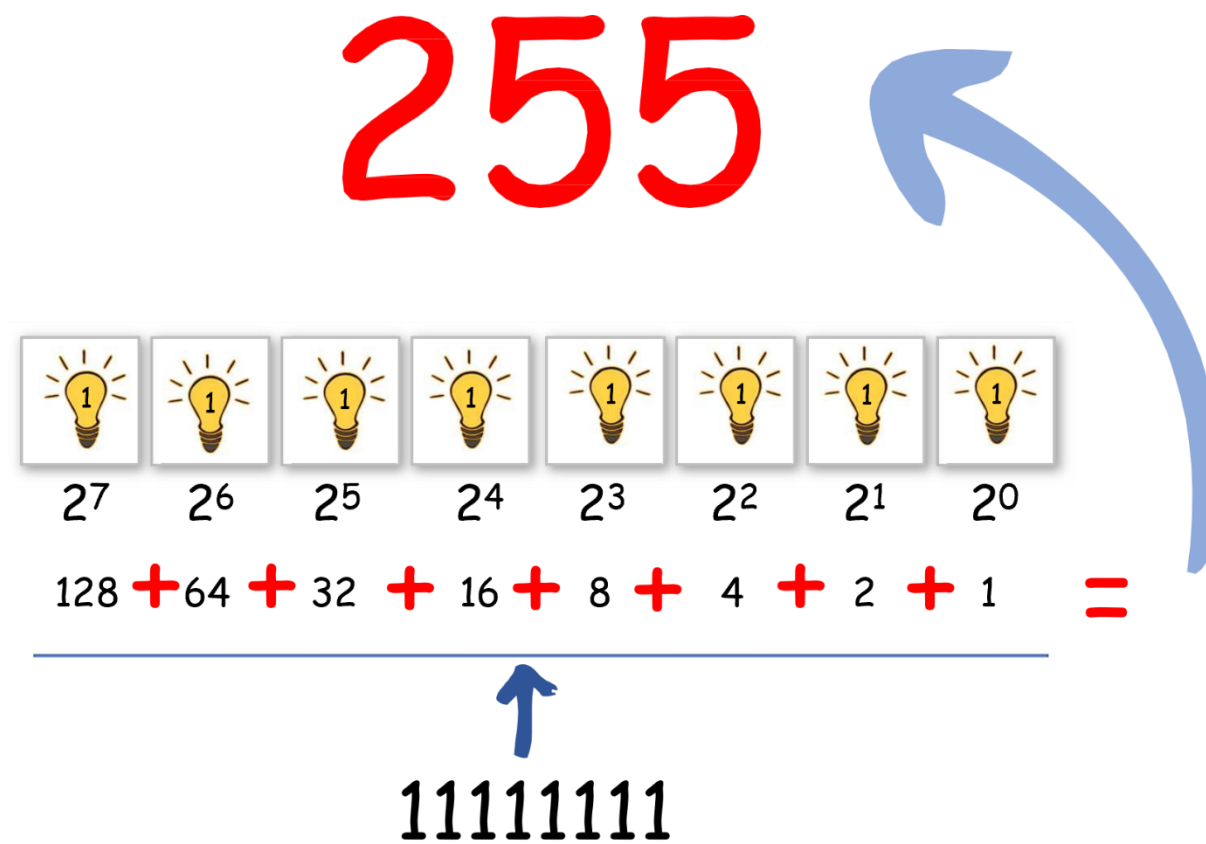
Programming Foundations

Level 1

Capacity of Byte

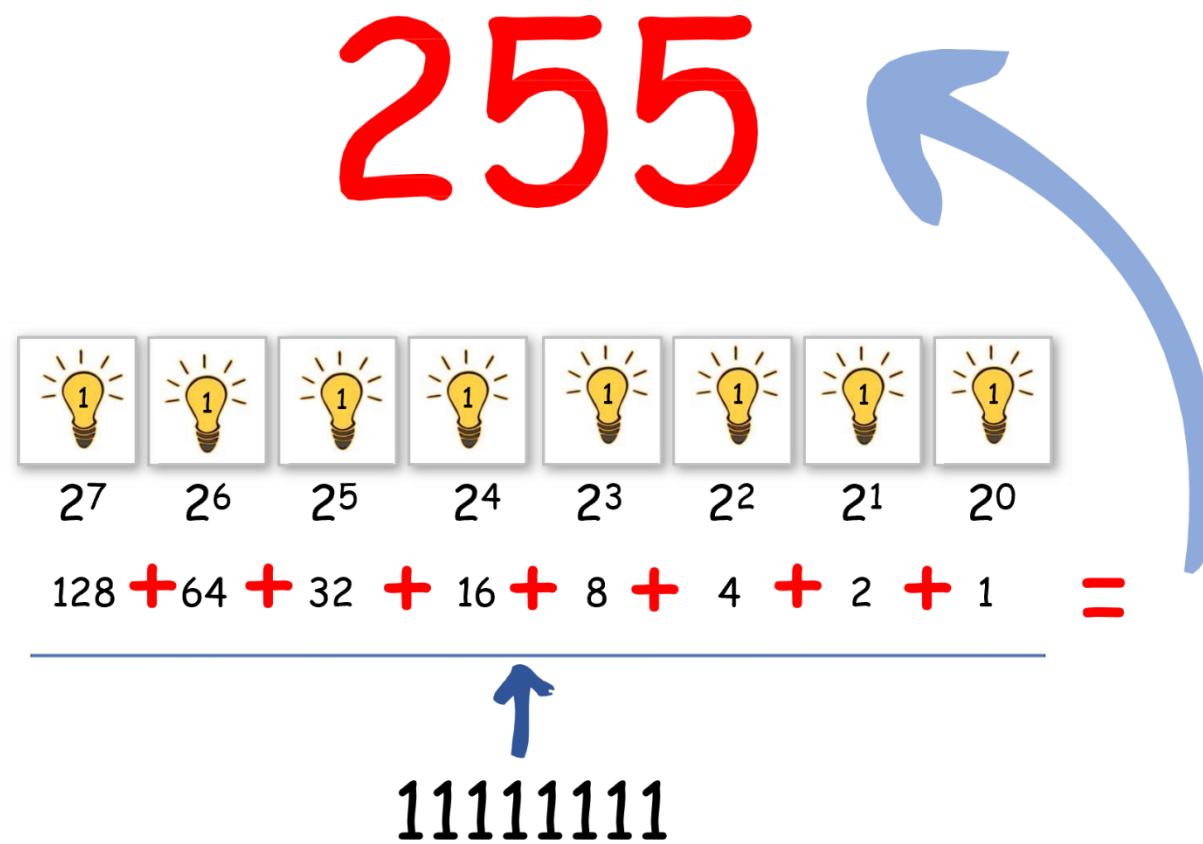
What is the largest number in byte?

1 byte = 8 bits → max number = 255 (11111111).



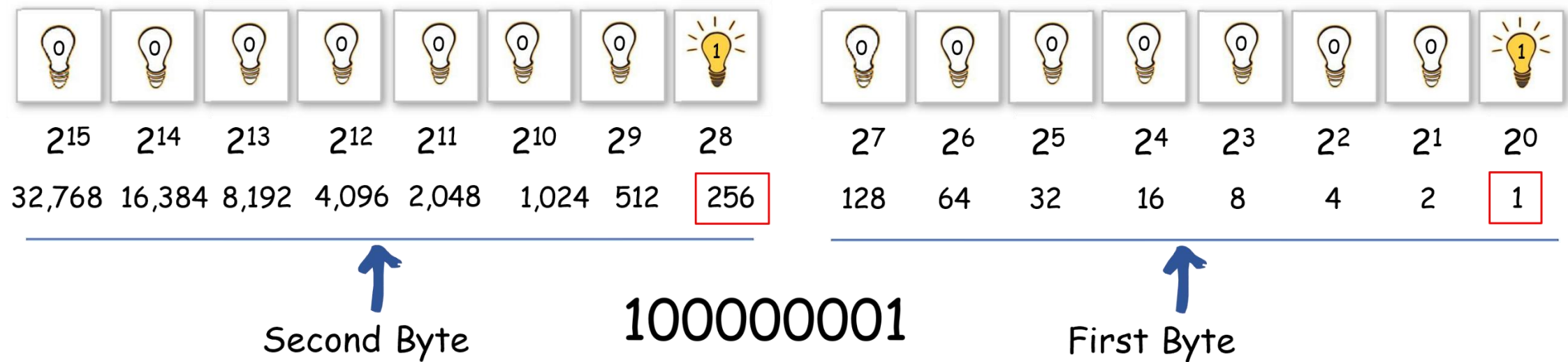
What if I have larger number such as 257?!!

2 bytes = 16 bits $\rightarrow$ max number = 65,535 (1111111111111111).



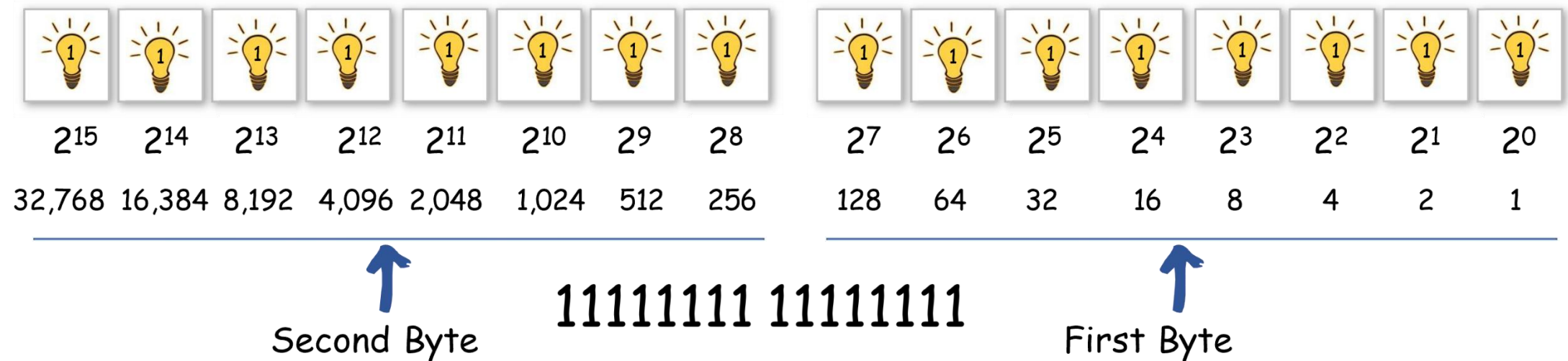
Represent this in binary...

257



What is the largest number in 2 bytes?

65,535



But!

How to represent letters?

The background of the slide features a complex, abstract pattern of thin, dark lines crisscrossing over a lighter, textured surface. In the lower portion of the image, the silhouettes of several people are visible, appearing to be in a collaborative setting, possibly a meeting or a workshop, with some individuals leaning forward as if working on a large display or table.

Programming Foundations

Level 1

ASCII & Unicode

Representing Characters - ASCII

- Computers use **codes** to represent letters and symbols.
- ASCII (American Standard Code for Information Interchange)** assigns a unique binary code to each character.
- Example:
 - 'A' = 65 = 01000001
 - 'B' = 66 = 01000010
 - Space = 32 = 00100000
- Limitation: ASCII supports only **English letters and limited symbols** (128 or 256 codes).

ASCII

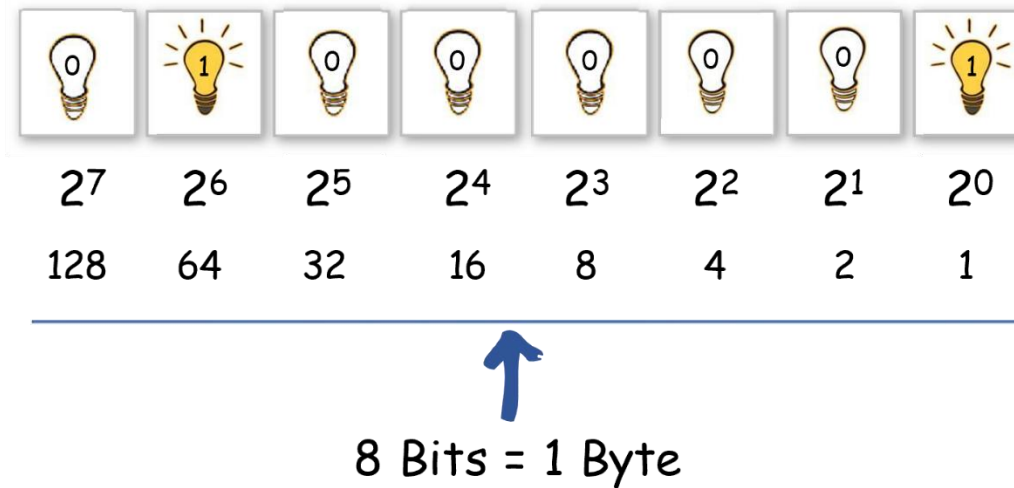
American Standard Code
for Information
Interchange

Letter	Binary	Decimal	Letter	Binary	Decimal
A	01000001	65	a	01100001	97
B	01000010	66	b	01100010	98
C	01000011	67	c	01100011	99
D	01000100	68	d	01100100	100
E	01000101	69	e	01100101	101
F	01000110	70	f	01100110	102
G	01000111	71	g	01100111	103
H	01001000	72	h	01101000	104
I	01001001	73	i	01101001	105
J	01001010	74	j	01101010	106
K	01001011	75	k	01101011	107
L	01001100	76	l	01101100	108
M	01001101	77	m	01101101	109
N	01001110	78	n	01101110	110
O	01001111	79	o	01101111	111
P	01010000	80	p	01110000	112
Q	01010001	81	q	01110001	113
R	01010010	82	r	01110010	114
S	01010011	83	s	01110011	115
T	01010100	84	t	01110100	116
U	01010101	85	u	01110101	117
V	01010110	86	v	01110110	118
W	01010111	87	w	01110111	119
X	01011000	88	x	01111000	120
Y	01011001	89	y	01111001	121
Z	01011010	90	z	01111010	122

How to represent letters?

65 Or A??

65  A



This is my name in binary:

Sara Alterkawi

<u>01010011</u>	<u>01100001</u>	<u>01110010</u>	<u>01100001</u>	<u>00100000</u>	<u>01000001</u>	<u>01101100</u>
<u>01110100</u>	<u>01100101</u>	<u>01110010</u>	<u>01101011</u>	<u>01100001</u>	<u>01110111</u>	<u>01101001</u>

What does this mean?

01001001 00100000 01001100 01101111 01110110 01100101 00100000
01011001 01101111 01110101 00100001



But!

How to represent Arabic letters?

The background of the slide is a dark, abstract image. It features a dense, intricate network of thin, light-colored lines that resemble a complex web or a neural network. In the lower portion of the image, there are dark silhouettes of several people. Two people are standing in the center, facing each other, while others are visible in the foreground, some appearing to be looking at a screen or device. The overall aesthetic is technical and modern.

Programming Foundations

Level 1

Unicode

Representing Characters - ASCII

■ To represent all world languages, **Unicode** was developed.

■ Unicode supports:

■ UTF-8 → 256 characters

■ UTF-16 → 65,536 characters

■ UTF-32 → 4,294,967,296 characters

💡 Example:

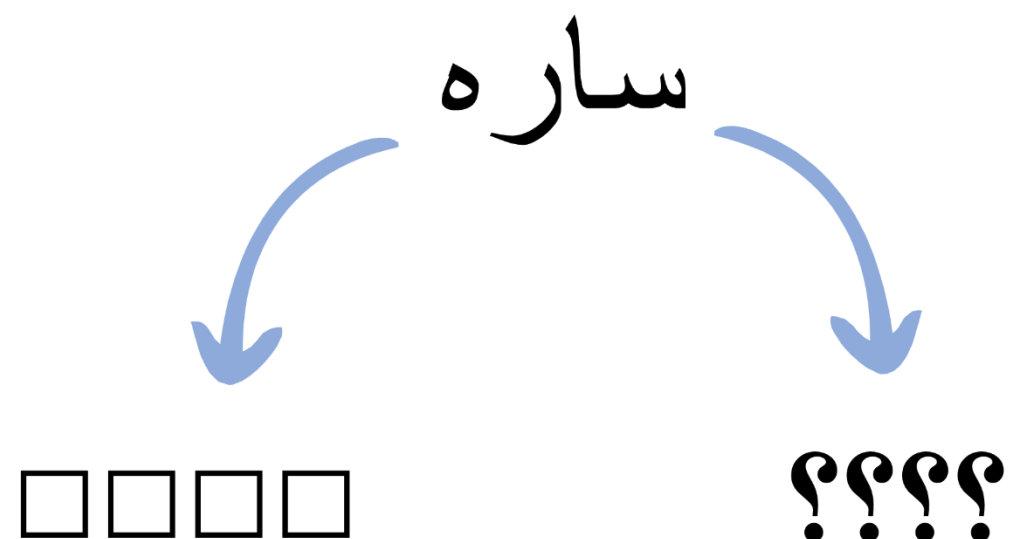
Arabic letters such as "ساره" require Unicode because ASCII cannot represent them.

ASCII

American Standard Code
for Information
Interchange

Letter	Binary	Decimal	Letter	Binary	Decimal
A	01000001	65	a	01100001	97
B	01000010	66	b	01100010	98
C	01000011	67	c	01100011	99
D	01000100	68	d	01100100	100
E	01000101	69	e	01100101	101
F	01000110	70	f	01100110	102
G	01000111	71	g	01100111	103
H	01001000	72	h	01101000	104
I	01001001	73	i	01101001	105
J	01001010	74	j	01101010	106
K	01001011	75	k	01101011	107
L	01001100	76	l	01101100	108
M	01001101	77	m	01101101	109
N	01001110	78	n	01101110	110
O	01001111	79	o	01101111	111
P	01010000	80	p	01110000	112
Q	01010001	81	q	01110001	113
R	01010010	82	r	01110010	114
S	01010011	83	s	01110011	115
T	01010100	84	t	01110100	116
U	01010101	85	u	01110101	117
V	01010110	86	v	01110110	118
W	01010111	87	w	01110111	119
X	01011000	88	x	01111000	120
Y	01011001	89	y	01111001	121
Z	01011010	90	z	01111010	122

Problem:



UNICODE

Unicode is an International
character encoding standard

Unicode is a **universal character encoding standard** that assigns a code to every character and symbol in every language in the world.

ASCII

VS

UNICODE

American Standard Code for Information Interchange

Universal Coding System

Supports specific characters (less space)

Large number of characters (large space)

Has 2 standards:
7-bit ASCII - 128 characters
8-bit ASCII - 256 characters

It has 3 standards:
UTF 8 - 256 characters
UTF 16 - 65,536 characters
UTF 32 - 4,294,967,296 characters

Only one language

Any combination of languages

Differences

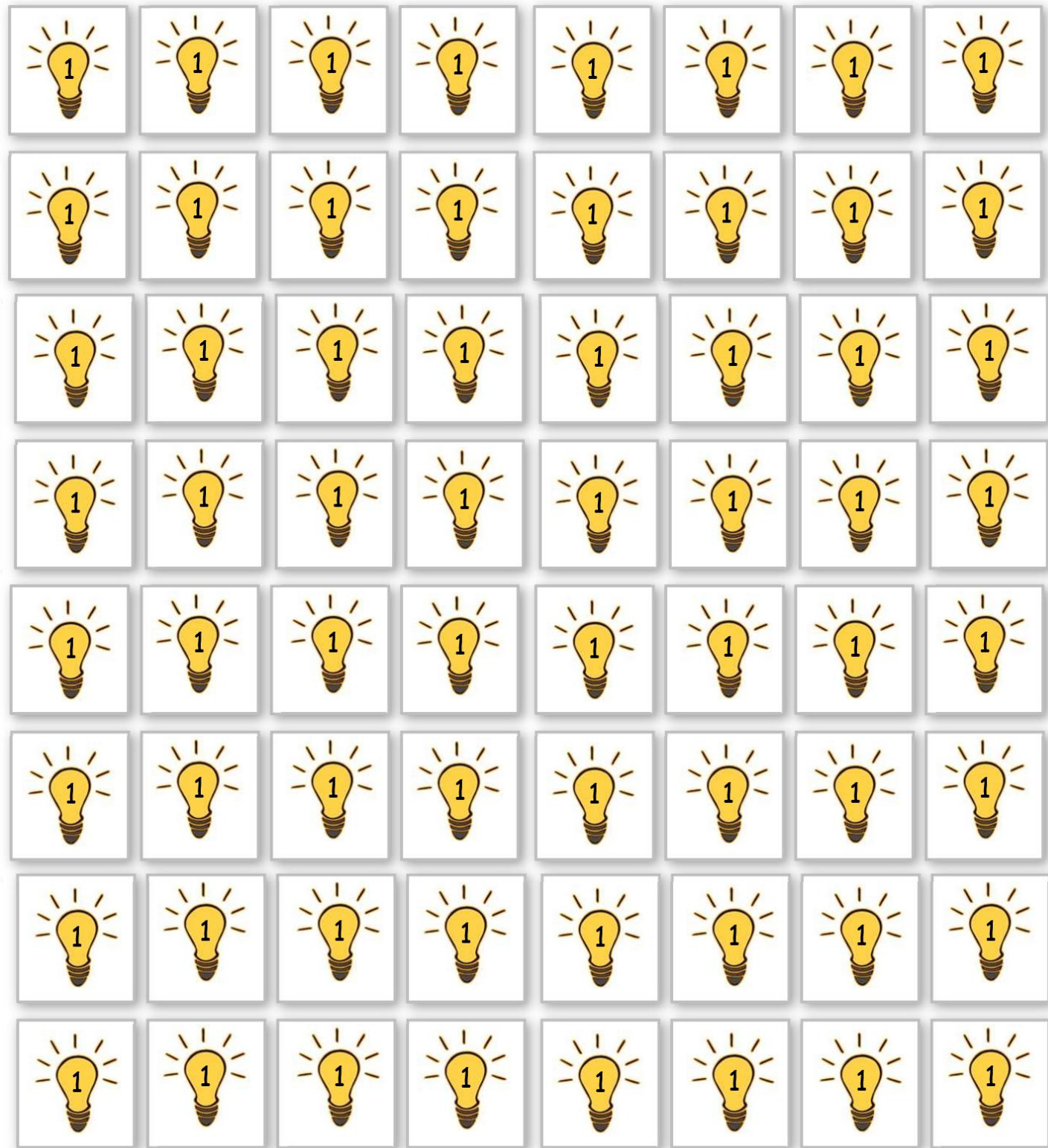
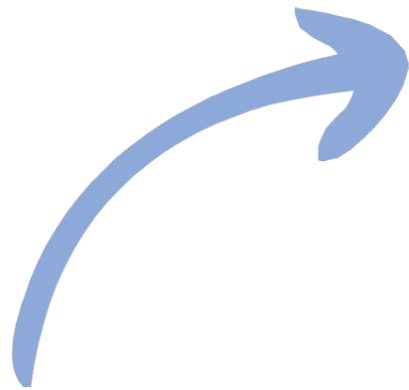
The background of the slide features a complex, abstract pattern of thin, dark lines crisscrossing over a lighter, textured surface. In the lower portion of the image, there are dark silhouettes of several people, some standing and some appearing to be in motion, which adds a sense of activity and collaboration to the scene.

Programming Foundations

Level 1

Largest number in Binary System

64 Bits = 8 Bytes



128	64	32	16	8	4	2	1
32,768	16,384	8,192	4,096	2,048	1,024	512	256
8,388,608	4,194,304	2,097,152	1,048,576	524,288	262,144	131,072	65,536
2,147,483,648	1,073,741,824	536,870,912	268,435,456	134,217,728	67,108,864	33,554,432	16,777,216
5.49756E+11	2.74878E+11	1.37439E+11	68,719,476,736	34,359,738,368	17,179,869,184	8,589,934,592	4,294,967,296
??							

8 bytes (64 bits) → Value on 64th square =
9,223,372,036,854,775,808.

128	64	32	16	8	4	2	1
32,768	16,384	8,192	4,096	2,048	1,024	512	256
8,388,608	4,194,304	2,097,152	1,048,576	524,288	262,144	131,072	65,536
2,147,483,648	1,073,741,824	536,870,912	268,435,456	134,217,728	67,108,864	33,554,432	16,777,216
5.49756E+11	2.74878E+11	1.37439E+11	68,719,476,736	34,359,738,368	17,179,869,184	8,589,934,592	4,294,967,296
??							

Total sum (1 to 64) → 18,446,744,073,709,600,000

🌟 Summary

- ✓ Binary System → Foundation for all digital data.
- ✓ Bits & Bytes → Determine data size and capacity.
- ✓ ASCII → Enables computers to represent text in English.
- ✓ Unicode → Expands capability to represent global languages.

By understanding how binary underlies all computer data, you gain insight into how programs, storage, and communication truly work! ⚡



Activity



Instructions:

1 Convert the following decimal numbers into binary:

44

129

512

717

999

2 Write your first name in binary using ASCII encoding

Activity Solution

 Solution:

1 Binary conversions:

 Convert 44 → Binary

Divide 44 by 2:

$44 \div 2 = 22$ remainder 0

$22 \div 2 = 11$ remainder 0

$11 \div 2 = 5$ remainder 1

$5 \div 2 = 2$ remainder 1

$2 \div 2 = 1$ remainder 0

$1 \div 2 = 0$, remainder 1

Now read remainders bottom → top:

1 0 1 1 0 0

 44 (decimal) = 101100 (binary)

Activity Solution

 Solution:

1 Binary conversions:

 Convert 129 → Binary

Divide 129 by 2:

$$129 \div 2 = 64 \text{ remainder } 1$$

$$64 \div 2 = 32 \text{ remainder } 0$$

$$32 \div 2 = 16 \text{ remainder } 0$$

$$16 \div 2 = 8 \text{ remainder } 0$$

$$8 \div 2 = 4 \text{ remainder } 0$$

$$4 \div 2 = 2 \text{ remainder } 0$$

$$2 \div 2 = 1 \text{ remainder } 0$$

$$1 \div 2 = 0 \text{ remainder } 1$$

Now read remainders bottom → top:

1 0 0 0 0 0 1

 129 (decimal) = 10000001 (binary)

Activity Solution

 Solution:

1 Binary conversions:

 Convert 512 → Binary

Divide 512 by 2:

$512 \div 2 = 256$ remainder 0

$256 \div 2 = 128$ remainder 0

$128 \div 2 = 64$ remainder 0

$64 \div 2 = 32$ remainder 0

$32 \div 2 = 16$ remainder 0

$16 \div 2 = 8$ remainder 0

$8 \div 2 = 4$ remainder 0

$4 \div 2 = 2$ remainder 0

$2 \div 2 = 1$ remainder 0

$1 \div 2 = 0$ remainder 1

Now read remainders bottom → top:

1 0 0 0 0 0 0 0 0 0

 512 (decimal) = 1000000000 (binary)

Activity Solution

 Solution:

1 Binary conversions:

 Convert 717 → Binary

Divide 717 by 2:

$717 \div 2 = 358$ remainder 1

$358 \div 2 = 179$ remainder 0

$179 \div 2 = 89$ remainder 1

$89 \div 2 = 44$ remainder 1

$44 \div 2 = 22$ remainder 0

$22 \div 2 = 11$ remainder 0

$11 \div 2 = 5$ remainder 1

$5 \div 2 = 2$ remainder 1

$2 \div 2 = 1$ remainder 0

$1 \div 2 = 0$ remainder 1

Now read remainders bottom → top:

1 0 1 0 0 1 1 0 1

 512 (decimal) = 101001101 (binary)

Activity Solution

 Solution:

1 Binary conversions:

 Convert 999 → Binary

Divide 999 by 2:

$999 \div 2 = 499$ remainder 1

$499 \div 2 = 249$ remainder 1

$249 \div 2 = 124$ remainder 1

$124 \div 2 = 62$ remainder 0

$62 \div 2 = 31$ remainder 0

$31 \div 2 = 15$ remainder 1

$15 \div 2 = 7$ remainder 1

$7 \div 2 = 3$ remainder 1

$3 \div 2 = 1$ remainder 1

$1 \div 2 = 0$ remainder 1

Now read remainders bottom → top:

1 1 1 1 1 0 0 1 1 1

 512 (decimal) = 1111100111 (binary)

Activity Solution

 Solution:

② Example (Name: "Omar"):

 Step 1 — Get the ASCII Codes

Letter	ASCII(Decimal)
O	79
m	109
a	97
r	114

Activity Solution

 Solution:

② Example (Name: "Omar"):

 Step 2: Now we convert **each** decimal value to binary using repeated division:

Convert "O" (79) to Binary

Divide 79 by 2:

$79 \div 2 = 39$ remainder 1

$39 \div 2 = 19$ remainder 1

$19 \div 2 = 9$ remainder 1

$9 \div 2 = 4$ remainder 1

$4 \div 2 = 2$ remainder 0

$2 \div 2 = 1$ remainder 0

$1 \div 2 = 0$ remainder 1

Now read remainders **bottom** → **top**:

1 0 0 1 1 1 1

 O = 01001111

Activity Solution

 Solution:

② Example (Name: "Omar"):

 Step 2: Now we convert **each** decimal value to binary using repeated division:

Convert "m" (109) to Binary

Divide 109 by 2:

$109 \div 2 = 54$	remainder 1
$54 \div 2 = 27$	remainder 0
$27 \div 2 = 13$	remainder 1
$13 \div 2 = 6$	remainder 1
$6 \div 2 = 3$	remainder 0
$3 \div 2 = 1$	remainder 1
$1 \div 2 = 0$	remainder 1

Now read remainders **bottom** → **top**:

1 1 0 1 1 0 1

 m = 01101101

Activity Solution

 Solution:

② Example (Name: "Omar"):

 Step 2: Now we convert **each** decimal value to binary using repeated division:

Convert "a" (97) to Binary

Divide 97 by 2:

$97 \div 2 = 48$ remainder 1

$48 \div 2 = 24$ remainder 0

$24 \div 2 = 12$ remainder 0

$12 \div 2 = 6$ remainder 0

$6 \div 2 = 3$ remainder 0

$3 \div 2 = 1$ remainder 1

$1 \div 2 = 0$ remainder 1

Now read remainders **bottom** → **top**:

1 1 0 0 0 0 1

 a = 01100001

Activity Solution

 Solution:

② Example (Name: "Omar"):

 Step 2: Now we convert **each** decimal value to binary using repeated division:

Convert "r" (114) to Binary

Divide 97 by 2:

$114 \div 2 = 57$ remainder 0

$57 \div 2 = 28$ remainder 1

$28 \div 2 = 14$ remainder 0

$14 \div 2 = 7$ remainder 0

$7 \div 2 = 3$ remainder 1

$3 \div 2 = 1$ remainder 1

$1 \div 2 = 0$ remainder 1

Now read remainders **bottom** → **top**:

1 1 1 0 0 1 0

 r = 01110010



Activity Solution



Solution:

2 Example (Name: "Omar"):



"Omar" = 01001111 01101101 01100001 01110010

Conclusion

- ✅ Binary is the *language of computers*. Every piece of data — from text to images is built from bits and bytes represented as 0s and 1s.
- ✅ Understanding binary helps you grasp how information is stored, processed, and communicated inside a computer.
- ✅ As a foundation in programming, mastering this concept prepares you for deeper topics like data types, memory, and algorithms. 🚀

Thank you 😊